

RIG Intelligence API — Documentation

Version: v1 **Base URL:** <https://api.your-domain.com/v1>

The RIG Intelligence API gives you programmatic access to structured media intelligence — briefings, sentiment, entities, stories, and geographic coverage — scoped entirely to your organization. Pull it into your own dashboards, apps, alerts, or reports.

You ask; you receive clean, finished intelligence as JSON. No setup, no infrastructure to run on your side.

Table of contents

1. [Getting started](#)
 2. [Authentication](#)
 3. [Your data scope](#)
 4. [Data freshness](#)
 5. [Endpoints - Briefings - Sentiment & analytics - Entities - Coverage \(articles & stories\) - Geography - Ask \(AI assistant\)](#)
 6. [Filtering — topics, entities & groups](#)
 7. [Response objects](#)
 8. [Pagination](#)
 9. [Rate limits & quotas](#)
 10. [Usage & billing](#)
 11. [Webhooks \(alerts\)](#)
 12. [Errors](#)
 13. [Security & isolation](#)
 14. [Interactive reference & support](#)
-

1. Getting started

When you onboard, you receive:

- **An API key** — your secret credential (treat it like a password).
- **Your base URL** — <https://api.your-domain.com/v1>.
- **A sandbox key** — a separate test key so you can build safely without touching production or your billing quota.

Your first call — fetch today's briefing:

```
curl -H "Authorization: Bearer YOUR_API_KEY" \
https://api.your-domain.com/v1/brief/today
```

Every endpoint returns JSON in a consistent envelope:

```
{
  "data": { ... },           // the result
  "meta": {                  // present on list endpoints
    "count": 20,
    "next_cursor": "eyJpZCI6..."
  }
}
```

Errors use:

```
{ "error": { "code": "unauthorized", "message": "Invalid API key", "status": 401 } }
```

2. Authentication

All requests are authenticated with your API key in the [Authorization](#) header as a Bearer token:

```
Authorization: Bearer YOUR_API_KEY
```

That single header is all you need — the API knows which organization you are from the key itself. You never pass an account or organization ID; it cannot be set, spoofed, or widened from the request.

Key management

- **Multiple keys** — issue separate keys for different apps/environments.
- **Rotate** — generate a new key and retire the old one at any time.
- **Revoke** — disable a key instantly if it's ever exposed.

Keys are managed from your account dashboard or by request to your account manager.

3. Your data scope

Your subscription covers an **agreed set of entities and topics** — your *scope*. Every response is automatically limited to that scope.

- You can query **any combination** of the entities and topics in your scope — one, several, or grouped with AND/OR logic (see [Filtering](#)).
- **Adding new entities or topics:** request them and we provision them to your scope.
- If it's something already covered, it's available **immediately**.
- If it's brand new to your scope, coverage **accumulates from that point forward**.

This keeps your feed focused — you only ever see what's relevant to you, and nothing outside your subscription.

4. Data freshness

Once a story is processed, it's available to you immediately through the API. Aggregated numbers — like sentiment trends or coverage counts — refresh on a short cycle, so those reflect the new story within about half an hour.

In practice:

You call	When the latest story shows up
Coverage / article endpoints	Immediately
Entity & story detail	Immediately
Analytics & aggregates (counts, sentiment trends)	Within ~30 minutes

Every call returns the current state — there is no nightly batch or stale export.

5. Endpoints

5.1 Briefings

GET /brief/today

Today's key developments across your scope.

```
curl -H "Authorization: Bearer YOUR_API_KEY" \
https://api.your-domain.com/v1/brief/today

{
  "data": {
    "date": "2026-06-29",
    "summary": "Three developments dominate your watchlist this week...",
    "sections": [
      {
        "title": "Water policy – rising criticism",
        "stories": [
          {
            "id": "sty_a1b2c3",
            "headline": "Opposition challenges new water allocation",
            "entities": [{ "id": "ent_551", "name": "Revanth Reddy", "type": "person" }],
            "sentiment": { "label": "critical", "intensity": 0.7 },
            "source": "The Hindu",
            "url": "https://www.thehindu.com/...",
            "published_at": "2026-06-29T08:30:00Z"
          }
        ]
      }
    ]
  }
}
```

GET /brief/daily?date=YYYY-MM-DD

The full digest for a given day (defaults to today). All sections, ranked.

GET /brief/situation

A rolling situation summary — the current state of play across your scope.

5.2 Sentiment & analytics

GET /analytics/sentiment

Sentiment toward an entity (or topic) over a time window.

Param	Type	Description
entity	string	Entity name or ID (required unless topic given)
topic	string	Topic name
window	int	Days to look back (default 7, max 90)

```
curl -H "Authorization: Bearer YOUR_API_KEY" \
  "https://api.your-domain.com/v1/analytics/sentiment?entity=Revanth%20Reddy&window=30"
```

```
{
  "data": {
    "subject": "Revanth Reddy",
    "window_days": 30,
    "total": 45,
    "split": { "supportive": 12, "neutral": 18, "critical": 15 },
    "net_lean": -0.07,
    "daily": [
      { "date": "2026-06-29", "supportive": 2, "neutral": 3, "critical": 2 },
      { "date": "2026-06-28", "supportive": 1, "neutral": 4, "critical": 3 }
    ]
  }
}
```

net_lean is a signed score from -1 (uniformly critical) to +1 (uniformly supportive). 0 is balanced.

GET /analytics/coverage?window=7

Article volume over time (great for trend lines).

```
{
  "data": {
    "window_days": 7,
    "total": 472,
    "daily": [ { "date": "2026-06-29", "count": 71 }, ... ]
  }
}
```

GET /analytics/outlets?entity=...&window=30

How each outlet is covering a subject — volume and lean per outlet.

```
{
  "data": {
    "subject": "Revanth Reddy",
    "outlets": [
      { "name": "The Hindu", "count": 12, "net_lean": 0.1 },

```

```
{
  { "name": "Eenadu", "count": 9, "net_lean": -0.3 }
}
```

GET /analytics/topics?window=7&limit=10

The most-covered topics in your scope for the window.

```
{
  "data": {
    "topics": [
      { "name": "water rights", "count": 88 },
      { "name": "law & order", "count": 54 }
    ]
  }
}
```

5.3 Entities

GET /entities

Your provisioned watchlist.

```
{
  "data": [
    { "id": "ent_551", "name": "Revanth Reddy", "type": "person" },
    { "id": "ent_777", "name": "Congress", "type": "party" }
  ]
}
```

GET /entities/{entity_id}

Profile of a single entity — name, type, and a rolling snapshot.

```
{
  "data": {
    "id": "ent_551",
    "name": "Revanth Reddy",
    "type": "person",
    "snapshot": {
      "coverage_7d": 45,
      "sentiment_7d": { "supportive": 12, "neutral": 18, "critical": 15 },
      "top_topics": ["water rights", "law & order"]
    }
  }
}
```

GET /entities/{entity_id}/coverage?window=7&limit=20

Coverage about this entity. Supports the full [filtering](#) and [pagination](#) options.

5.4 Coverage (articles & stories)

GET /articles

The main filterable feed. Returns individual coverage items.

Param	Type	Description
entity	string (repeatable)	Filter to one or more entities
topic	string (repeatable)	Filter to one or more topics
match	any all	Combine multiple filters with OR (any, default) or AND (all)
sentiment	supportive neutral critical	Only items with this tone
from / to	date	Date range (ISO YYYY-MM-DD)
language	string	Language code (e.g. en, te, hi)
sort	recent relevant	Order (default recent)
limit	int	Page size (default 20, max 100)
cursor	string	Pagination cursor

```
curl -H "Authorization: Bearer YOUR_API_KEY" \
  "https://api.your-domain.com/v1/articles?entity=Revanth%20Reddy&topic=water&match=all&sentiment=critical&w
```

```
{
  "data": [
    {
      "id": "art_12345",
      "headline": "Opposition challenges new water allocation",
      "summary": "The opposition criticised the government's revised...",
      "entities": [{ "id": "ent_551", "name": "Revanth Reddy", "type": "person" }],
      "sentiment": { "label": "critical", "intensity": 0.8 },
      "topic": "water rights",
      "geo": { "state": "Telangana", "district": "Hyderabad" },
      "source": "The Hindu",
      "language": "en",
      "url": "https://www.thehindu.com/...",
      "published_at": "2026-06-29T09:15:00Z",
      "story_id": "sty_a1b2c3"
    }
  ],
  "meta": { "count": 20, "next_cursor": "eyJpZCI6..." }
}
```

GET /articles/{article_id}

A single coverage item in full.

GET /stories

Stories group related coverage into a single thread (one evolving event, many articles). Same filter params as `/articles`.

```
{
  "data": [
    {
      "id": "sty_a1b2c3",
      "title": "Water allocation dispute",
      "article_count": 23,
      "entities": [{ "id": "ent_551", "name": "Revanth Reddy", "type": "person" }],
      "sentiment": { "supportive": 4, "neutral": 8, "critical": 11 },
      "first_seen": "2026-06-25T06:00:00Z",
      "last_updated": "2026-06-29T09:15:00Z"
    }
  ]
}
```

`GET /stories/{story_id}`

A story in full — its timeline, the articles in it, entities, and sentiment breakdown.

5.5 Geography

`GET /geo/coverage?window=7`

Coverage volume by state/district across your scope.

```
{
  "data": {
    "states": [
      { "state": "Telangana", "count": 220,
        "districts": [ { "district": "Hyderabad", "count": 88 } ] }
    ]
  }
}
```

`GET /geo/district/{district_id}?window=7&limit=20`

Coverage for a specific district. Supports filtering and pagination.

5.6 Ask (AI assistant)

`POST /ask`

Ask a natural-language question; receive a synthesized, **source-cited** answer streamed back over Server-Sent Events (`text/event-stream`). It can also return charts (e.g. a sentiment breakdown) inline.

Request

```
{ "query": "How has sentiment toward the CM shifted this week?", "history": [] }
```

Stream events

type	Meaning
------	---------

<code>status</code>	Progress update (e.g. "searching")
<code>token</code>	A chunk of the answer text
<code>chart</code>	A chart to render (labels + series)
<code>sources</code>	The coverage items the answer is grounded in
<code>done</code>	Stream complete

```
const res = await fetch('https://api.your-domain.com/v1/ask', {
  method: 'POST',
  headers: { 'Authorization': `Bearer ${API_KEY}`, 'Content-Type': 'application/json' },
  body: JSON.stringify({ query: 'How is sentiment toward the CM this week?', history: [] })
});

const reader = res.body.getReader();
const decoder = new TextDecoder();
while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  for (const line of decoder.decode(value).split('\n')) {
    if (!line.startsWith('data:')) continue;
    const ev = JSON.parse(line.slice(5));
    if (ev.type === 'token') process.stdout.write(ev.text);
    if (ev.type === 'chart') renderChart(ev);
    if (ev.type === 'sources') showSources(ev.articles);
  }
}
```

Every answer cites the specific coverage it used, so you can always trace a statement back to its source.

6. Filtering — topics, entities & groups

The coverage endpoints (`/articles`, `/stories`, `/entities/{id}/coverage`, `/geo/...`) share one consistent filter language.

Single filter

```
/articles?entity=Revanth%20Reddy
/articles?topic=water
```

Multiple values (repeat the param)

```
/articles?entity=Revanth%20Reddy&entity=KCR
/articles?topic=water&topic=power
```

Groups + AND/OR with `match` - `match=any` (default) → OR. Any of the filters can match. - `match=all` → AND. All must match the same item.

```
# Coverage mentioning (Revanth OR KCR) AND about (water OR power):
/articles?entity=Revanth&entity=KCR&topic=water&topic=power&match=all
```

Combine with tone, dates, language


```
/articles?entity=Revanth&sentiment=critical&from=2026-06-01&to=2026-06-29&language=te
```

All filters are restricted to your provisioned scope — if you reference an entity or topic outside it, the API returns an empty result (and you can [request it added](#)).

7. Response objects

Coverage item (article)

Field	Type	Description
<code>id</code>	string	Unique ID
<code>headline</code>	string	The headline
<code>summary</code>	string	Short synopsis
<code>entities</code>	array	<code>{ id, name, type }</code> for each entity mentioned
<code>sentiment</code>	object	<code>{ label, intensity }</code> — tone toward the subject
<code>topic</code>	string	Primary topic
<code>geo</code>	object	<code>{ state, district }</code>
<code>source</code>	string	Publishing outlet
<code>language</code>	string	Language code
<code>url</code>	string	Link to the original
<code>published_at</code>	string	ISO 8601 timestamp
<code>story_id</code>	string	The story thread it belongs to

Sentiment object

Field	Type	Description
<code>label</code>	string	<code>supportive</code> <code>neutral</code> <code>critical</code>
<code>intensity</code>	float	<code>0–1</code> , how strong the tone is

Entity object

Field	Type	Description
<code>id</code>	string	Unique ID
<code>name</code>	string	Display name
<code>type</code>	string	<code>person</code> <code>party</code> <code>organization</code> <code>place</code>

Responses contain only these business fields — clean and stable. We won't change a field's meaning under you; new fields may be added over time.

8. Pagination

List endpoints are **cursor-paginated**.

- Pass `limit` (default `20`, max `100`).
- The response `meta.next_cursor` is the cursor for the next page.
- When `next_cursor` is `null`, you've reached the end.

```
let cursor = null;
do {
  const url = new URL('https://api.your-domain.com/v1/articles');
  url.searchParams.set('entity', 'Revanth Reddy');
  url.searchParams.set('limit', '100');
  if (cursor) url.searchParams.set('cursor', cursor);

  const res = await fetch(url, { headers: { 'Authorization': `Bearer ${API_KEY}` } });
  const { data, meta } = await res.json();
  handle(data);
  cursor = meta.next_cursor;
} while (cursor);
```

9. Rate limits & quotas

Each key has a request rate limit and a monthly quota (set by your plan).

Every response includes:

```
X-RateLimit-Limit: 600
X-RateLimit-Remaining: 587
X-RateLimit-Reset: 1719655200
```

If you exceed the rate limit you get `429 Too Many Requests` with a `Retry-After` header (seconds to wait):

```
{ "error": { "code": "rate_limited", "message": "Too many requests", "status": 429, "retry_after": 30 } }
```

Back off and retry after `retry_after` seconds. Need higher limits? Ask your account manager.

10. Usage & billing

GET /usage

See your current period's consumption against your quota.

```
{
  "data": {
    "period": "2026-06",
    "requests": 14820,
    "quota": 50000,
    "remaining": 35180,
    "by_endpoint": [
      { "endpoint": "/articles", "requests": 9100 },
      { "endpoint": "/analytics/sentiment", "requests": 3200 }
    ]
  }
}
```

Usage is metered transparently so there are no billing surprises.

11. Webhooks (alerts)

Get pushed a notification the moment new coverage matches a filter — instead of polling.

POST /webhooks

```
{
  "url": "https://your-app.com/hooks/rig",
  "filter": { "entity": "Revanth Reddy", "sentiment": "critical" }
}
```

When matching coverage appears, we **POST** to your URL:

```
{
  "event": "coverage.matched",
  "article": { "id": "art_998", "headline": "...", "sentiment": { "label": "critical", "intensity": 0.9 }, "
}
```

Manage subscriptions with **GET /webhooks** and **DELETE /webhooks/{id}**. Each delivery is signed so you can verify it came from us.

12. Errors

Standard HTTP status codes, with a JSON body.

Status	code	Meaning
400	bad_request	Malformed parameters

401	unauthorized	Missing or invalid API key
403	forbidden	Key valid but not permitted for this resource
404	not_found	Resource doesn't exist or is outside your scope
429	rate_limited	Rate limit exceeded — see retry_after
500	server_error	Something went wrong on our side — retry shortly

```
{ "error": { "code": "bad_request", "message": "Unknown parameter 'windwo'", "status": 400 } }
```

Always check the status code before parsing [data](#).

13. Security & isolation

- **TLS everywhere** — all traffic is encrypted in transit.
- **Strict tenant isolation** — every request is scoped to your organization on our servers. You can only ever access your own data; there is no parameter, header, or trick that can widen access to another client's information.
- **Scoped, rotatable keys** — revoke or rotate at any time; compromise of a key never exposes anyone else.
- **Least-privilege responses** — endpoints return only the documented business fields.

14. Interactive reference & support

- **Interactive API reference (Swagger/OpenAPI):** <https://api.your-domain.com/v1/docs> — try every endpoint live with your sandbox key, and download the OpenAPI spec to generate a client in your language.
- **Sandbox:** use your sandbox key against the same endpoints to build and test freely.
- **Support:** integration@your-domain.com or your account manager.

RIG Intelligence API - v1 - © RIG360 Media